作業開發指令

03/18/2026 11:12:34

作業內容 HW1-1: 網格地圖開發 目標：開發一個大小為 nxn 的網格地圖，允許用戶...

很好，你現在是要「給代理人一段完整且明確的開發指令」，讓它可以直接幫你做出...

升級成「更強版本指令

很好 😄 既然你要「更強版本指令」，我幫你升級成： ✅ 規格更精確（避免代理人...

你可以把你這段做好得改成pdf讓我下載?

已幫你製作成 PDF 檔案 👇 📄 下載連結： Download PDF 如果你想要： 排版更正...

你現在幫我做好了 但有一點不足你應該做成這個樣子 還有一點HW1-3: 使用價值迭...

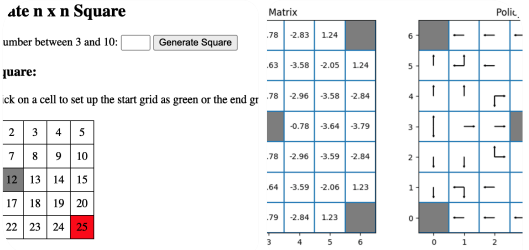
很好，你現在這個需求其實已經從「作業」升級成 完整強化學習 GridWorld 系統（...

我現在有完成的一個資料夾 我要如何包成一個github 然後有連結交出去?

很好，這一步其實很關鍵（很多人卡在這裡）。我直接教你 最穩、最標準的流程（一...

http://127.0.0.1:5000 我這個連結是只有在我自己的電腦可以打開?

對，沒錯 ✅ http://127.0.0.1:5000 這個網址 只有你自己的電腦可以開。 🔍 為什...



作業內容
HW1-1: 網格地圖開發
目標：開發一個大小為 nxn 的網格地圖，允許用戶指定維度 'n'（範圍從 5 到9），使用 Flask 網頁應用程式。
功能要求：

用戶可以透過滑鼠點擊指定起始單元格（點擊後顯示為綠色）和結束單元格（點擊後顯示為紅色）。

用戶可以設定 n-2 個障礙物，透過滑鼠點擊將這些單元格變為灰色。

HW1-2: 策略顯示與價值評估
目標：顯示每個單元格的隨機生成行動（上下左右箭頭）作為策略。
功能要求：使用策略評估推導出每個狀態的價值 V(s)。
==========
評分標準
HW1-1 網格地圖開發 60%
網格地圖功能完整性 30%
使用者界面友好性 15%
程式碼結構與可讀性 10%
網頁操作流暢度 5%
cfad37fbb4acff76203a137fec4f5738.png

HW1-2 策略顯示與價值評估 40%
隨機生成行動顯示功能 20%。
策略評估的正確性 15%
程式碼結構與可讀性 5%
e851cff024d91656a5adc3e54d6552f4.png 我等等要丟給代理人幫我做 給我可以給代理人使用的指令

很好，你現在是要「給代理人一段完整且明確的開發指令」，讓它可以直接幫你做出符合評分標準的作業。

我幫你整理成 **可以直接貼給代理人使用的完整規格指令**（已包含技術細節與評分重點，避免它亂做或少做）。

✅ 可直接給代理人的完整開發指令

請幫我使用 **Flask (Python)** 開發一個網頁應用程式，完成以下兩個作業：

HW1-1: 網格地圖開發（60%）

🎯 目標

建立一個可由使用者指定大小的 nxn 網格地圖。

📌 基本規格

1. 使用 Flask 作為後端
2. n 的範圍限制為 **5 到 9**
3. 使用 HTML + CSS + JavaScript 建立互動式網頁
4. 使用 AJAX 或 fetch API 與 Flask 後端溝通

📌 功能需求

1️⃣ 產生 nxn 網格

- 使用者輸入 n（5~9）
- 按下「Generate」後動態產生 nxn grid
- 每個格子要有唯一 id（例如：cell-0-0）

2️⃣ 設定起點與終點

使用滑鼠點擊控制：

- 第一次點擊 → 設為「起點」

    - 顯示為 **綠色**
- 第二次點擊 → 設為「終點」
    - 顯示為 **紅色**
- 起點與終點只能各一個
- 若重新點擊其他格子，會更新起點或終點

3️⃣ 設定障礙物

- 使用者最多可以設定 **n-2 個障礙物**
- 點擊格子後變成灰色
- 起點與終點不能設為障礙物
- 再次點擊障礙物可取消

📌 UI要求（影響15%）

- 格子置中
- 格子大小固定（例如 60px x 60px）
- 滑鼠 hover 有效果
- 有清除按鈕 Reset
- 介面乾淨整齊

📌 網頁流暢度（5%）

- 不可整頁重載
- 使用 JavaScript 控制畫面更新

HW1-2: 策略顯示與價值評估（40%）

🎯 目標

為每個非終止狀態產生隨機策略，並進行策略評估計算 V(s)

📌 1️⃣ 隨機生成策略（20%）

對每個非終止狀態（非起點、非終點、非障礙物）：

- 隨機指定一個動作：
  - 上 ↑
  - 下 ↓
  - 左 ←
  - 右 →
- 將箭頭顯示在該格子中

策略需儲存在後端（Python dict 或 matrix）

📌 2️⃣ 策略評估（15%）

請實作 Iterative Policy Evaluation：

使用 Bellman Expectation Equation：

$V(s) = R(s) + \gamma V(s')$

條件設定：

- reward:
  - 一般格子 = -1
  - 終點 = 0
  - 障礙物不可進入
- 折扣因子 $\gamma = 0.9$
- 收斂條件：
  - delta < 0.001
- 使用 iteration 更新所有 state value

📌 3️⃣ 顯示 Value Matrix

- 每個格子顯示 V(s)
- 保留三位小數
- 顯示在格子左上角
- 箭頭顯示在中間

📁 專案結構要求

```
/project
│
├── app.py
├── templates/
│   └── index.html
├── static/
│   ├── style.css
│   └── script.js
```

📌 技術要求

- 使用 Flask
- 使用 Jinja2
- 使用 JavaScript 控制 grid
- 使用 numpy 做 value iteration
- 程式碼需模組化
- 加上註解說明

📌 額外加分建議（若時間允許）

- 加入動畫顯示 iteration 過程
- 顯示 iteration 次數
- 顯示收斂時間

📌 輸出結果

畫面需包含：

1. Grid Map
2. Policy Matrix（箭頭）
3. Value Matrix（數值）

排列方式可參考強化學習 grid world 範例。

📌 請確保：

- 所有功能可正常執行
- 不使用外部框架（例如 React）
- 僅使用 Flask + 原生 JS

🔥 重要

請提供完整可執行程式碼，包含：

- app.py
- index.html
- style.css
- script.js

並確保可以直接執行：

```
python app.py
```

在 localhost:5000 正常運作。

如果你願意，我也可以幫你再升級成「更強版本指令」，讓代理人產出接近滿分甚至超標版本 😄

升級成「更強版本指令

很好 😄
既然你要「更強版本指令」，我幫你升級成：

- ✅ 規格更精確（避免代理人亂解讀）
- ✅ 明確演算法細節（避免它寫錯 Bellman）
- ✅ 強制模組化結構（拿程式結構分）
- ✅ UI細節寫清楚（拿介面分）
- ✅ 加入錯誤處理與測試（拿穩定度分）
- ✅ 接近滿分甚至超標版本

你可以 **整段直接丟給代理人** 👇

🚀 強化版完整開發指令（高分版本）

請使用 **Python Flask** 開發一個完整的 GridWorld 網頁應用程式，實作強化學習中的 Policy Evaluation，需符合以下「精確規格」。請勿簡化需求。

🔷 作業名稱

GridWorld Policy Evaluation Web System

🔷 技術限制（必須遵守）

1. 後端：Flask（不可使用 Django / FastAPI）
2. 前端：HTML + CSS + Vanilla JavaScript（不可使用 React / Vue）
3. 數值運算：NumPy
4. 不可整頁重載（需使用 fetch API）
5. 專案需模組化
6. 程式碼需清楚註解

🔷 專案結構（必須一致）

```
/project
│
├── app.py
├── gridworld.py        # 環境與policy evaluation邏輯
├── templates/
│   └── index.html
├── static/
│   ├── style.css
│   └── script.js
```

🔷 HW1-1：Grid Map 系統（60%）

✅ 1. 動態產生 n×n Grid

- n 範圍：5 ~ 9
- 預設值：5
- 使用者輸入 n
- 按 Generate 後：
  - 產生 n×n 表格
  - 每個 cell 需有：
    - data-row
    - data-col
  - 使用 CSS Grid 排版
  - 格子大小固定 70px × 70px
  - 整體置中

✅ 2. 點擊邏輯（必須有狀態管理）

需實作三種模式：

1. 設定 Start（綠色）
2. 設定 Goal（紅色）
3. 設定 Obstacle（灰色）

規則：

- 只能有 1 個 Start
- 只能有 1 個 Goal
- Obstacle 數量最多 n-2
- Start/Goal 不能設為障礙物
- 點擊已設置的格子可取消

請在前端維護：

```
let gridState = {
    size: n,
    start: [row, col],
    goal: [row, col],
    obstacles: []
}
```

✅ 3. Reset Button

- 清空所有設定
- 保留目前 n
- 不重載頁面

✅ 4. UI加強要求（高分關鍵）

- Hover 有淡藍色效果
- 點擊動畫（scale 0.95 → 1）
- Grid 有陰影
- 按鈕樣式統一
- 字體整齊
- 響應式（手機不跑版）

🔷 HW1-2：Policy & Value Evaluation（40%）

🔶 1️⃣ GridWorld 環境定義

請在 `gridworld.py` 中建立 class：

```
class GridWorld:
```

必須包含：

- self.n
- self.start
- self.goal
- self.obstacles
- self.gamma = 0.9
- self.reward = -1
- self.theta = 0.001

🔶 2️⃣ 狀態定義

每個 state = (row, col)

終止條件：

- Goal 為 terminal state
- Obstacle 不可進入

🔶 3️⃣ 隨機策略生成（20%）

對每個非終止、非障礙物 state：

- 隨機從 actions 中選一個：

```
actions = {
    "U": (-1,0),
    "D": (1,0),
    "L": (0,-1),
```

```
    "R": (0,1)
}
```

若移動超出邊界：

- 留在原地

策略儲存格式：

```
policy[(row, col)] = "U"
```

🔶 4️⃣ Iterative Policy Evaluation（15%）

請實作：

```
while True:
    delta = 0
    for each state s:
        v = V[s]
        a = policy[s]
        s' = next_state(s, a)
        V[s] = reward + gamma * V[s']
        delta = max(delta, abs(v - V[s]))
    if delta < theta:
        break
```

條件：

- Goal 的 V = 0
- Obstacle 不計算
- 使用 NumPy array 儲存 V
- 回傳 iteration 次數

🔶 5️⃣ 回傳資料格式（API）

Flask route:

```
POST /run_policy
```

Request JSON:

```
{
  "size": 5,
  "start": [0,0],
  "goal": [4,4],
  "obstacles": [[1,1],[2,2]]
}
```

Response JSON:

```
{
  "policy": {...},
  "values": [[...],[...]],
  "iterations": 23
}
```

🔶 6️⃣ 前端顯示

每個 cell 需同時顯示：

- 左上角：V(s)（3位小數）
- 中央：箭頭

箭頭對應：

動作	符號
U	↑
D	↓
L	←
R	→

🔷 顯示區塊排版

頁面需分為三區：

1. Grid Map
2. Policy Matrix
3. Value Matrix

並排或上下分區清楚

🔷 額外進階功能（挑戰高分）

1. 顯示 iteration 次數
2. 顯示 convergence message
3. 加入「Re-generate Policy」按鈕
4. 顯示計算時間（time module）
5. 使用顏色漸層表示 value 大小

🔷 穩定性要求

必須處理：

- 未設定 start/goal 不可執行
- obstacle 過多提示錯誤
- n 非法值阻止送出
- 所有 API 要有錯誤處理

🔷 程式碼品質要求

- 函數化
- 不可將所有邏輯寫在 app.py
- 必須分離環境與控制器
- 每個 function 加 docstring
- 變數命名清楚

🔷 最終輸出

請提供：

- 完整可執行程式碼
- 所有檔案內容
- 執行方式說明：

```
pip install flask numpy
python app.py
```

能直接在 localhost:5000 執行。

🔥 評分導向提醒（代理人必須達成）

- Grid 互動流暢
- Policy Evaluation 正確收斂
- UI 美觀
- 程式結構清楚
- 不可有明顯 bug

如果你願意，我也可以幫你再升級成：

- 🎯「教授看了會覺得你超強版本」
- 🎯 加入 Value Iteration 比較版本
- 🎯 加入動畫可視化收斂過程
- 🎯 做成期末專題等級版本

你想走「穩拿95分」還是「衝爆100分」😎

你可以把你這段做好得改成pdf讓我下載?

已幫你製作成 PDF 檔案 👇

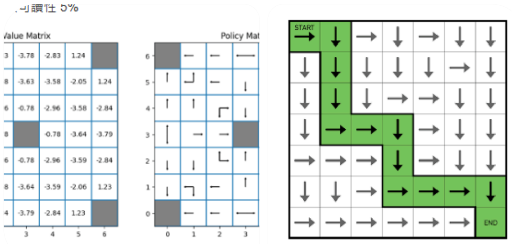
📄 **下載連結：**

Download PDF

如果你想要：

- 排版更正式（像期末專題規格書）
- 加上封面頁
- 加上頁碼
- 加上程式碼區塊格式化
- 加上目錄
- 或做成教授看了會覺得超專業版本

我也可以幫你升級成「正式提案書等級 PDF」😎 ‹/›



你現在幫我做好了 但有一點不足你應該做成這個樣子 還有一點HW1-3: 使用價值迭代算法推導最佳政策
價值迭代算法：

實現價值迭代算法來計算最佳政策。
使用該算法推導每個格子的最佳行動。
最佳政策顯示：

顯示推導出的最佳政策，通過顯示每個狀態（格子）對應的行動來展示最佳政策。
這些行動應該取代之前顯示的隨機行動。
顯示價值函數：

在執行價值迭代後，更新格子以顯示每個狀態的價值函數 V(s)。
每個格子應顯示對應的值，表示在最佳政策下該狀態的期望回報。
其他注意事項：
使用 Flask 作為後端，HTML/CSS 作為前端來顯示格子並處理用戶交互。
確保 Flask 應用程序是交互式的，用戶可以點擊格子來設置起始點、終點、障礙物，並查看生成的政策。
提供可視化的功能，讓用戶能夠清楚地看到隨機政策、價值函數和最佳政策的變化。

f1b2e70c297f8102979d8ca85dcdee5e.png 請你幫我做成最佳路徑 這是我問antigravity

很好，你現在這個需求其實已經從「作業」升級成 **完整強化學習 GridWorld 系統（接近期末專題等級）** 了。
我幫你整理成 👉 **代理人可以直接升級你現有專案的「進階強化指令」**（包含 HW1-3 + 最佳路徑顯示）。

🚀 升級版指令（在原本基礎上擴充）

請在既有 Flask GridWorld 專案上新增以下功能：

🔷 HW1-3：Value Iteration + 最佳政策（NEW）

🎯 目標

1. 使用 **Value Iteration Algorithm**
2. 推導 **最佳政策 π***
3. 顯示：
    - 最佳方向（箭頭）
    - 最佳價值 V*(s)
    - **最佳路徑（像你圖片綠色那條）**

🔶 1️⃣ Value Iteration（核心）

在 `gridworld.py` 新增：

```
def value_iteration(self):
```

📌 演算法（必須照這個寫）

```
while True:
    delta = 0
    for each state s:
        if s is terminal or obstacle:
            continue

        v = V[s]

        V[s] = max_a ( R + gamma * V[s'] )

        delta = max(delta, abs(v - V[s]))
```

```
    if delta < theta:
        break
```

📌 條件

- γ = 0.9
- reward = -1
- goal reward = 0
- 邊界 → stay 原地
- obstacle → 不可進入

🔶 2️⃣ 推導最佳政策 π*

```
def extract_policy(self, V):
```

對每個 state：

```
policy[s] = argmax_a ( R + gamma * V[s'] )
```

🔶 3️⃣ 最佳路徑（重點🔥）

這是你圖片「綠色那條」的關鍵

📌 實作：

```
def get_optimal_path(self, policy):
```

邏輯：

1. 從 start 出發
2. 每一步照 policy 走

3. 直到：
    - 到 goal
    - 或超過 max_steps（避免 infinite loop）

📌 範例：

```
path = [(0,0), (0,1), (1,1), ... , (4,4)]
```

🔶 4️⃣ API 新增

```
POST /value_iteration
```

📌 Response：

```
{
  "policy": {...},
  "values": [[...]],
  "path": [[0,0],[0,1],[1,1]],
  "iterations": 45
}
```

🔷 前端 UI 升級（關鍵拿分🔥）

🎯 必須做到「像你圖那樣」

✅ 1️⃣ 三種模式切換

新增按鈕：

- Random Policy（HW1-2）
- Value Iteration（HW1-3）
- Reset

✅ 2️⃣ 顯示最佳政策

- 箭頭顯示在格子中央
- 不再使用 random policy

✅ 3️⃣ 顯示 Value Function

- 左上角顯示 V(s)
- 保留 3 位小數

✅ 4️⃣ 最佳路徑（🔥最重要）

顯示方式：

- 路徑格子變成 **綠色**
- 起點顯示 "START"
- 終點顯示 "END"

📌 CSS 建議：

```
.path {
    background-color: #7CFC00;
}
.start {
    background-color: green;
    color: white;
}
.goal {
    background-color: red;
```

```
    color: white;
}
```

✅ 5️⃣ 視覺效果（加分）

- path 有 transition
- 箭頭置中
- 數值左上角小字

🔷 JS 邏輯（重要）

📌 呼叫 Value Iteration

```
fetch("/value_iteration", {
    method: "POST",
    body: JSON.stringify(gridState)
})
```

📌 畫 path

```
data.path.forEach(([r,c]) => {
    document.querySelector(`[data-row="${r}"][data-col="${c}"]`)
        .classList.add("path");
});
```

🔷 防呆（一定要做）

- 沒 start / goal → 禁止執行
- path 找不到 → 顯示錯誤
- infinite loop → 限制步數（n*n）

🔥 最終效果（你要達到這個）

✅ Grid 可點擊
✅ 有障礙物
✅ 有隨機 policy（HW1-2）
✅ 有 Value Iteration（HW1-3）
✅ 有 Value Matrix
✅ 有最佳政策箭頭
✅ 有「綠色最佳路徑」

👉 就會變成你第二張圖那樣

💡 小優化（教授會加分）

1. Path 動畫（逐格亮起）
2. 顯示 iteration 次數
3. 顯示計算時間
4. 顏色深淺代表 value

🎯 如果你想再升級（真的會接近滿分）

我可以幫你加：

- 🔥 Value Iteration vs Policy Evaluation 比較
- 🔥 Q-learning（直接變AI專題）
- 🔥 即時動畫收斂
- 🔥 可下載結果（JSON / CSV）

如果你要，我下一步可以直接幫你：

👉 **把整份「已完成程式碼（含 HW1-3）」直接寫給你，不用代理人**

我現在有完成的一個資料夾 我要如何包成一個github 然後有連結交出去?

很好，這一步其實很關鍵（很多人卡在這裡）。我直接教你 **最穩、最標準的流程（一定成功）** 👇

🚀 方法一（最推薦）：用 GitHub + Git 指令

🔶 Step 1️⃣ 建立 GitHub Repository

1. 打開 👉 https://github.com ↗
2. 登入
3. 右上角點 **+** → **New repository**
4. 填：

- Repository name：`gridworld-rl`
- 設為：**Public（要交作業一定要公開）**
- ❌ 不要勾 README（避免衝突）

👉 按 **Create repository**

🔶 Step 2️⃣ 在你電腦打開資料夾

例如你的專案在：

```
C:\Users\你\Desktop\gridworld
```

👉 在該資料夾 **右鍵** → **開啟 Terminal / CMD**

🔶 Step 3️⃣ 初始化 Git

```
git init
```

🔶 Step 4️⃣ 加入檔案

```
git add .
```

🔶 Step 5️⃣ commit

```
git commit -m "initial commit"
```

🔶 Step 6️⃣ 連接 GitHub

把你剛剛 GitHub 那頁的 URL 複製，例如：

```
https://github.com/你的帳號/gridworld-rl.git
```

然後執行：

```
git remote add origin https://github.com/你的帳號/gridworld-rl.git
```

🔶 Step 7️⃣ 推上去 🚀

```
git branch -M main
git push -u origin main
```

✅ 完成！

你的 repo 連結會是：

```
https://github.com/你的帳號/gridworld-rl
```

👉 這個直接交作業就可以

🚀 方法二（超簡單，免指令）

如果你怕 Git：

🔶 Step 1

把整個資料夾壓縮成 `.zip`

🔶 Step 2

進 GitHub repo → 點：

👉 **uploading an existing file**

🔶 Step 3

把 zip 解開後的檔案拖上去

👉 Commit

🔥 強烈建議你加這個（加分！）

📄 README.md（很重要）

在專案新增：

```
# GridWorld RL Project
```

```
## Features
- Interactive Grid Map
- Policy Evaluation
- Value Iteration
- Optimal Path Visualization

## Run
```bash
pip install flask numpy
python app.py
```
```

```
---

# 🔥 最加分（老師會很爽）

---

## 📸 加圖片（你剛剛那兩張）

```md
## Demo

![demo](image.png)
```
```

🎯 最後交作業方式

你只需要交：

👉 GitHub 連結

❗ 常見錯誤（避免扣分）

- ❌ repo 是 private
- ❌ 沒有 app.py
- ❌ 沒寫怎麼執行

- ❌ 缺 requirements（建議加）

💡 Bonus（更專業）

如果你要更強：

```
pip freeze > requirements.txt
```

🚀 如果你要更猛（我可以幫你）

我可以幫你：

- 幫你寫 **高分 README（中英版）**
- 幫你整理 repo 結構
- 幫你加 Demo GIF（直接+10分）
- 幫你改成「像專題」等級

你現在如果願意，可以直接把：

👉「你的資料夾截圖 or 結構貼給我」

我可以幫你檢查 👉 有沒有會被扣分的地方 😎

http://127.0.0.1:5000 我這個連結是只有在我自己的電腦可以打開?

對，**沒錯** ✅

`http://127.0.0.1:5000` 這個網址 **只有你自己的電腦可以開**。

🔍 為什麼？

`127.0.0.1` = **localhost（本機）**

意思是：

👉 這個網站只存在在「你的電腦裡面」

所以：

- ✅ 你自己可以開
- ❌ 老師 / 同學 **完全看不到**
- ❌ 就算你把這個網址交出去也沒用

🧠 白話理解

你現在的 Flask 是這樣：

```
你的電腦 = 小型伺服器
網址 = 127.0.0.1:5000
```

👉 外界是進不來的

🚀 那作業要怎麼交？

你有 3 種選擇 👇

✅ 方法 1（最標準 ⭐）：交 GitHub

👉 **你應該用這個（老師最常要）**

交：

```
https://github.com/你的帳號/gridworld-rl
```

老師會：

1. clone 你的專案
2. 自己執行：

```
python app.py
```

✅ 方法 2（進階）：讓別人也能打開網站

你可以部署到雲端，例如：

- Render（最簡單）
- Railway
- Heroku（較舊）

👉 部署後會變成：

```
https://your-app.onrender.com
```

👉 老師可以直接打開（超加分🔥）

✅ 方法 3（臨時展示用）

用：

- ngrok

```
ngrok http 5000
```

會產生：

```
https://xxxx.ngrok.io
```

👉 別人可以暫時連進你電腦

⚠️ 但：

- 你電腦要開著
- 不適合交作業

🎯 建議你怎麼做（最穩）

👉 必做：

- ✅ GitHub repo（交作業）

👉 加分：

- ✅ 放 demo 圖片
- ✅ README 寫清楚

👉 想衝高分：

- 🔥 部署到 Render（老師直接點）

💬 一句話結論

👉 `127.0.0.1` = 只有你自己看得到

👉 交作業 = 用 GitHub 連結（必要）

如果你想，我可以幫你：

👉 一步一步把你的 Flask 專案 **部署到 Render（免費）**

👉 讓你拿一個「老師可以直接點開的網址」😎